

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Design Pattern III

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Design pattern parte III



Observer pattern

Observer pattern 1

L'**Observer pattern** (o "Pattern Osservatore") è un **pattern di progettazione comportamentale** che stabilisce una modalità per gestire le dipendenze tra oggetti, in modo che un cambiamento di uno stato di un oggetto ne notifichi e ne aggiorni automaticamente molti altri.



Definisce un canale di **comunicazione uno-a-molti** tra gli oggetti, in modo che quando lo stato di un oggetto (chiamato **Subject, Observable** o **Producer**) cambia, tutti i suoi osservatori (chiamati **Observers** o **Subscribers**) vengano avvisati e aggiornati automaticamente.

Instaurando una comunicazione, permette di **disaccoppiare (eliminare dipendenze)** tra interi package di classi.

Observer pattern 2

💬 Scopo

Observer è un pattern di progettazione comportamentale che ti permette di definire un meccanismo di sottoscrizione per notificare oggetti multipli riguardo a qualsiasi evento che accada all'oggetto che stanno osservando.



😞 Problema

Immagina un software **Misuratore** che controlla dei dispositivi di misurazione di alcune variabili meteorologiche (ad es. temperatura e pressione). Esistono poi diversi dispositivi **Monitor** per monitorare tali variabili ad esempio quelli che si possono comprare ed appendere al muro del proprio studio.

Come far aggiornare questi dispositivi **Monitor**?

Ogni volta che qualcuno acquista un dispositivo Monitor, devo modificare il codice del **Misuratore** e cablare il riferimento a questi oggetti **Monitor** nel codice?

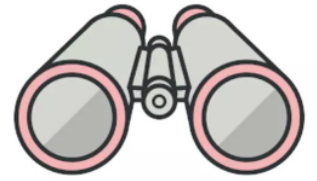
Devo quindi interrompere l'esecuzione, modificare il codice e riavviare tutto? IMPOSSIBILE AVERE QUESTA DIPENDENZA !!!



Observer pattern 3

😊 Soluzione / Ricetta (in Java)

Behavioral

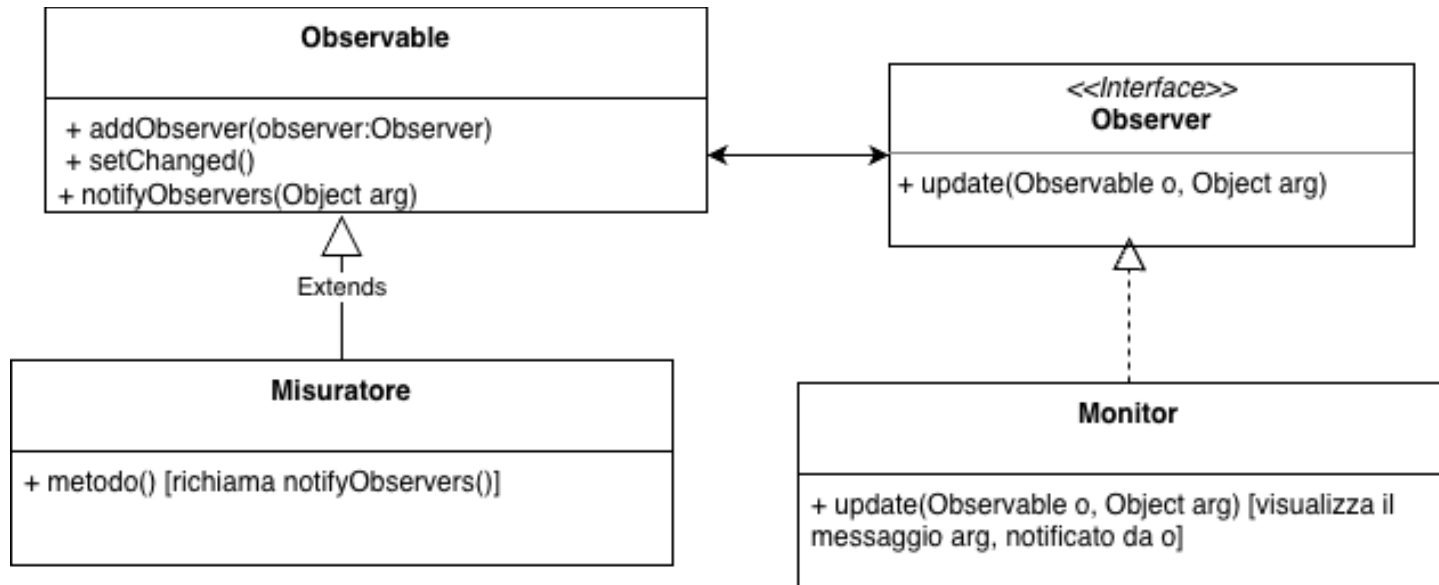


Observer



Observer pattern 4

Esempio/Schema



Misuratore e Monitor sono classi che non hanno idea dell'esistenza l'una dell'altra. (sono completamente disaccoppiate)

- 1) Per rendere una Classe Osservabile estendo **Observable** e richiamo `setChanged()` e `notifyObservers()` ogni volta che cambia lo stato.
- 2) Per rendere una classe Osservatrice:
 - implemento **Observer** e il metodo `update()`
- 3) Qualcuno attiverà la comunicazione tramite:
`observable.addObserver(observer);`

Observer in Java è Deprecato

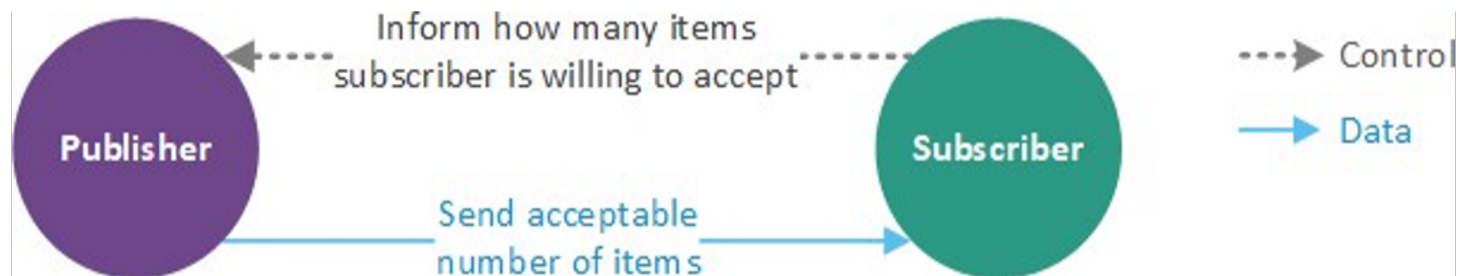
Per il Progetto potete (e vi consiglio di) usarli ANCHE SE deprecati.

Il problema di questo pattern è che non c'è alcun controllo sulla quantità di messaggi che possono partire dagli osservabili.

In Java esiste un altro pattern non deprecato "REACTIVE STREAM"

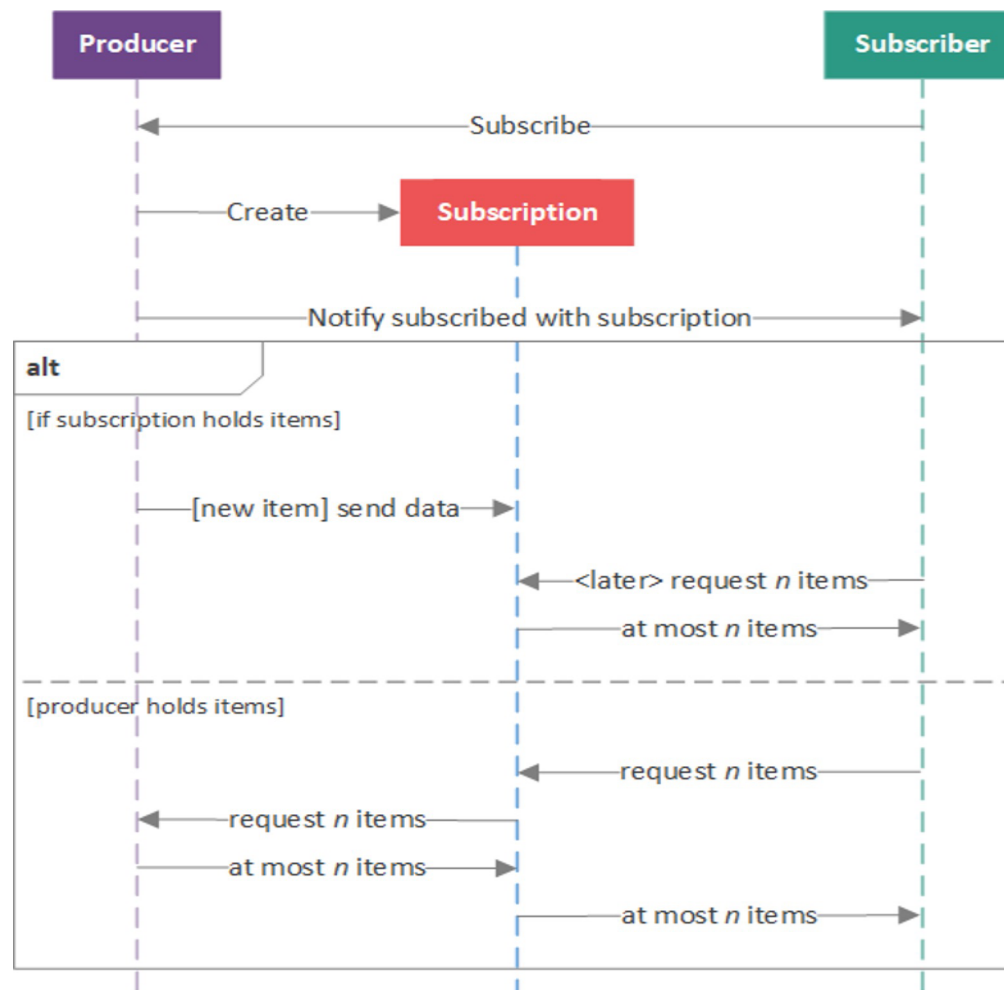
Reactive Stream (Flow) 1

- Gestisce il recupero di dati da sorgenti multiple, (quasi) in tempo reale, con continui invii di informazione
- L'invio di informazione è tipicamente asincrono
- Non sappiamo in anticipo quando i dati saranno inviati



Reactive Stream (Flow) 2

Sono coinvolti 4 attori: Producer, Subscriber, Subscription e Processor



Reactive Stream (Flow) 3

Se un'entità è sia un Producer che un Subscriber, è un Processor

- Un intermediario nello stream di dati
- Permette di concatenare più elaborazioni dei dati in una sorta di **pipeline**



Reactive Stream (Flow) 4

- Da java 9 Quattro interfacce definite all'interno della classe `java.util.concurrent.Flow`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static interface	<code>Flow.Processor<T,R></code>	A component that acts as both a Subscriber and Publisher.
static interface	<code>Flow.Publisher<T></code>	A producer of items (and related control messages) received by Subscribers.
static interface	<code>Flow.Subscriber<T></code>	A receiver of messages.
static interface	<code>Flow.Subscription</code>	Message control linking a <code>Flow.Publisher</code> and <code>Flow.Subscriber</code> .

- `Flow.Publisher` ha un solo metodo:

Method Summary

All Methods

Instance Methods

Abstract Methods

Modifier and Type	Method	Description
void	<code>subscribe(Flow.Subscriber<? super T> subscriber)</code>	Adds the given Subscriber if possible.

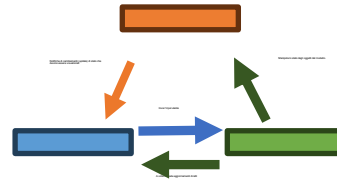
Reactive Stream (Flow) 5

- `Flow.Subscriber<T>` ha 4 metodi:

All Methods	Instance Methods	Abstract Methods
Modifier and Type	Method	Description
void	<code>onComplete()</code>	Method invoked when it is known that no additional Subscriber method invocations will occur for a Subscription that is not already terminated by error, after which no other Subscriber methods are invoked by the Subscription.
void	<code>onError(Throwable throwable)</code>	Method invoked upon an unrecoverable error encountered by a Publisher or Subscription, after which no other Subscriber methods are invoked by the Subscription.
void	<code>onNext(T item)</code>	Method invoked with a Subscription's next item.
void	<code>onSubscribe(Flow.Subscription subscription)</code>	Method invoked prior to invoking any other Subscriber methods for the given Subscription.

- `Flow.Processor<T,R>` estende `Subscriber<T>` e `Producer<R>`
- `Flow.Subscription` ne ha 2:

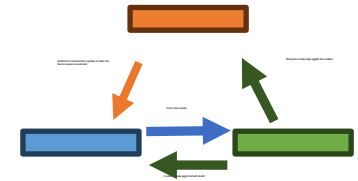
All Methods	Instance Methods	Abstract Methods
Modifier and Type	Method	Description
void	<code>cancel()</code>	Causes the Subscriber to (eventually) stop receiving messages.
void	<code>request(long n)</code>	Adds the given number n of items to the current unfulfilled demand for this subscription.



Model View Controller (architetturale)

MVC 1

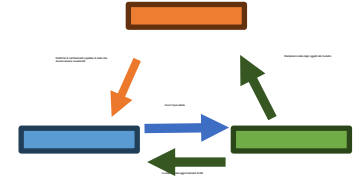
MVC (Model-View-Controller) è un pattern di progettazione software comunemente utilizzato per progettare applicazioni dotate di interfacce utente, dati e logica di controllo.



Esso enfatizza una separazione tra la logica di business del software (model) e la sua visualizzazione (view).

Questa "separazione delle preoccupazioni" (o responsabilità) [separation of concerns] fornisce una migliore divisione del lavoro e una manutenzione migliorata.

MVC 2



💬 Scopo

Separazione delle responsabilità Logica (Modello) e Interfaccia Utente (Vista).

Permette un riuso al 100% delle classi che si occupano della logica di business (Modello).

😞 Problema

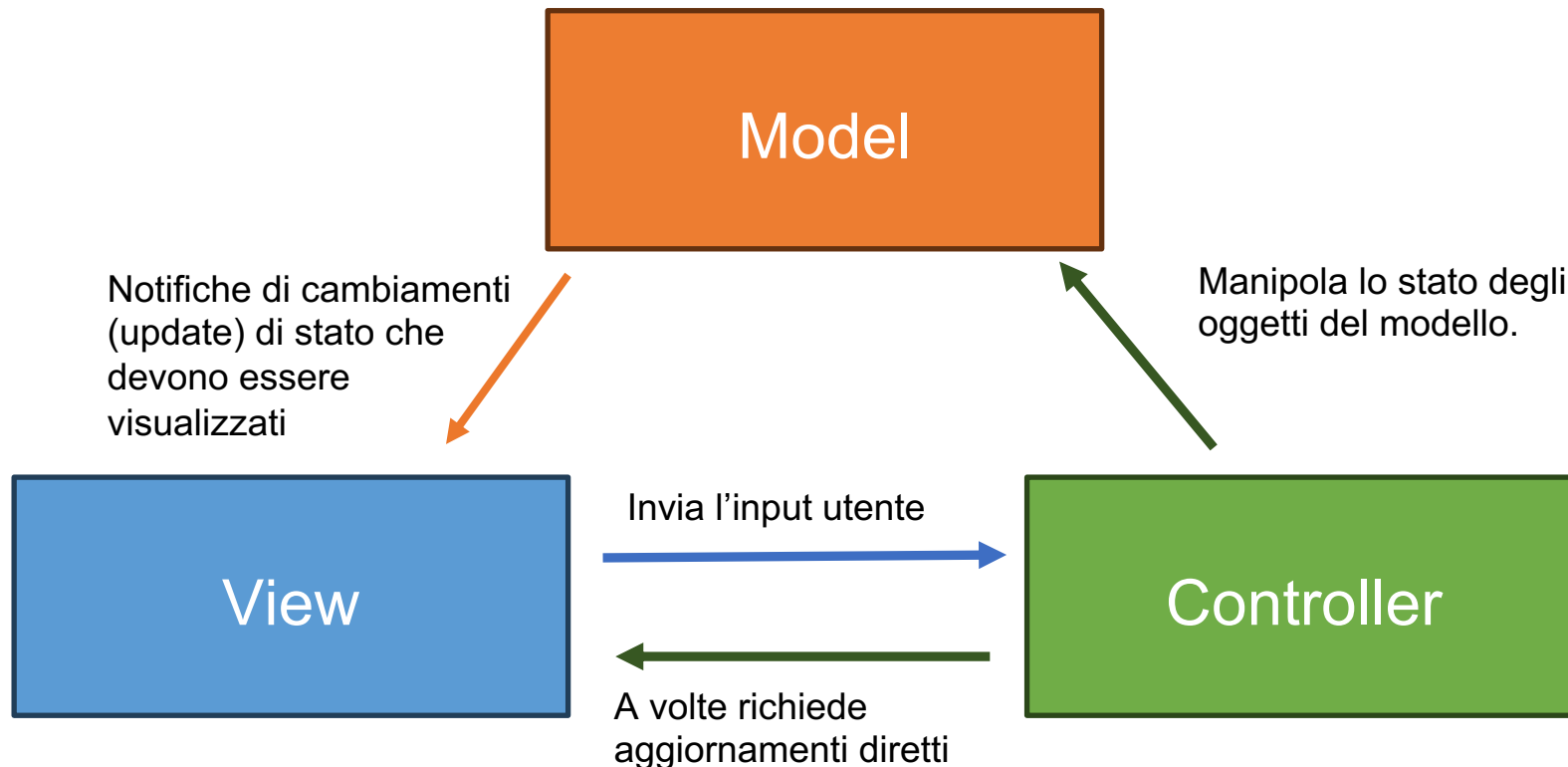
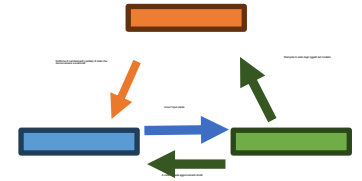
Immaginiamo di aver scritto un software (ad es. un videogioco).

Il software ha un enorme successo e abbiamo l'opportunità di «venderlo» su diverse piattaforme.

Tuttavia il codice mischia la gestione della logica con dell'interfaccia utente, il processo di **porting** sulle altre piattaforme, che prevedono altre tecnologie per l'interfaccia utente (ad es. librerie, motori grafici etc.), consiste di fatto nella riscrittura COMPLETA di tutto il programma.

MVC 3

😊 Soluzione / Ricetta

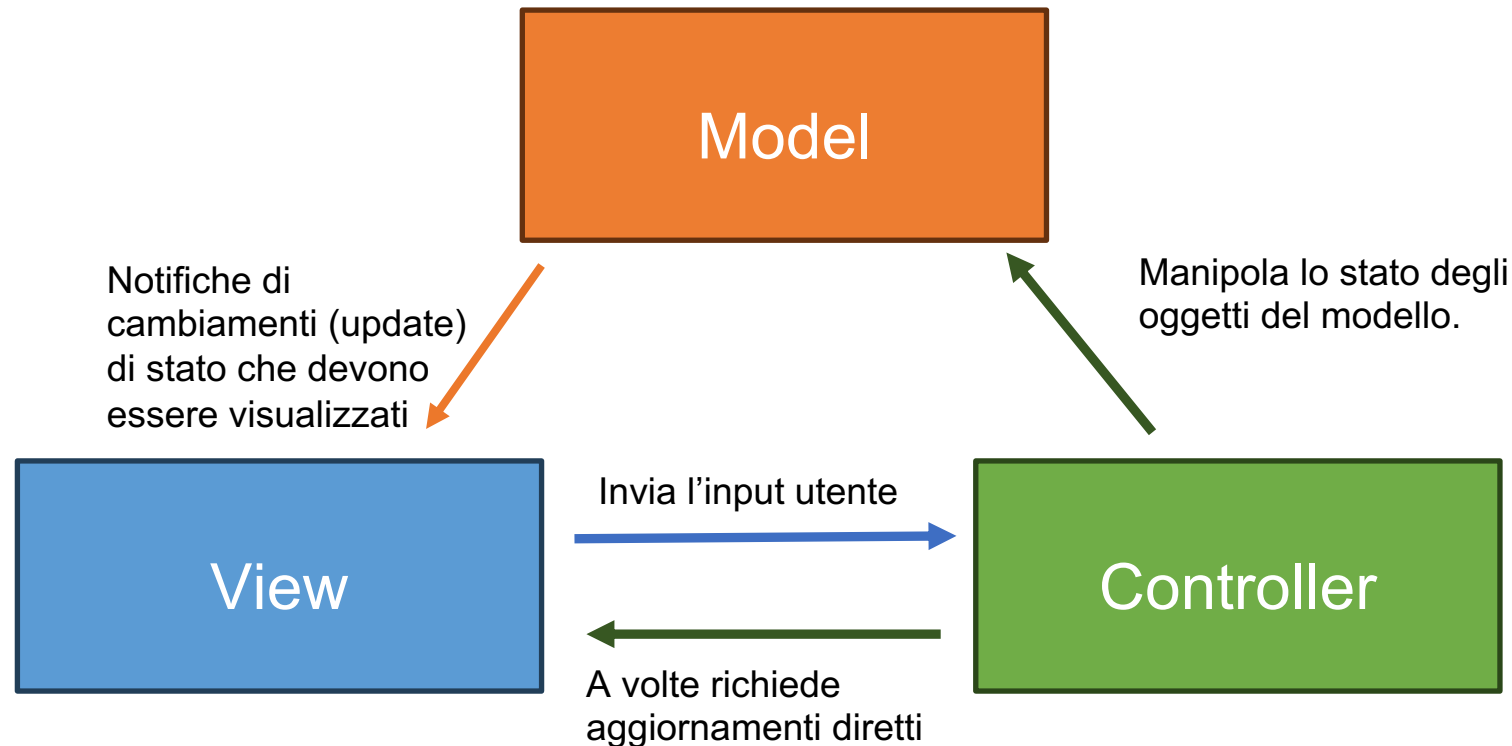


Le tre parti del pattern di progettazione software MVC possono essere descritte come segue:

- **Model (Modello):** Gestisce i dati e la logica di business.
- **View (Vista):** Gestisce il layout e la visualizzazione.
- **Controller (Controllore):** Instrada i comandi alle parti del Modello e della Vista.

MVC 4

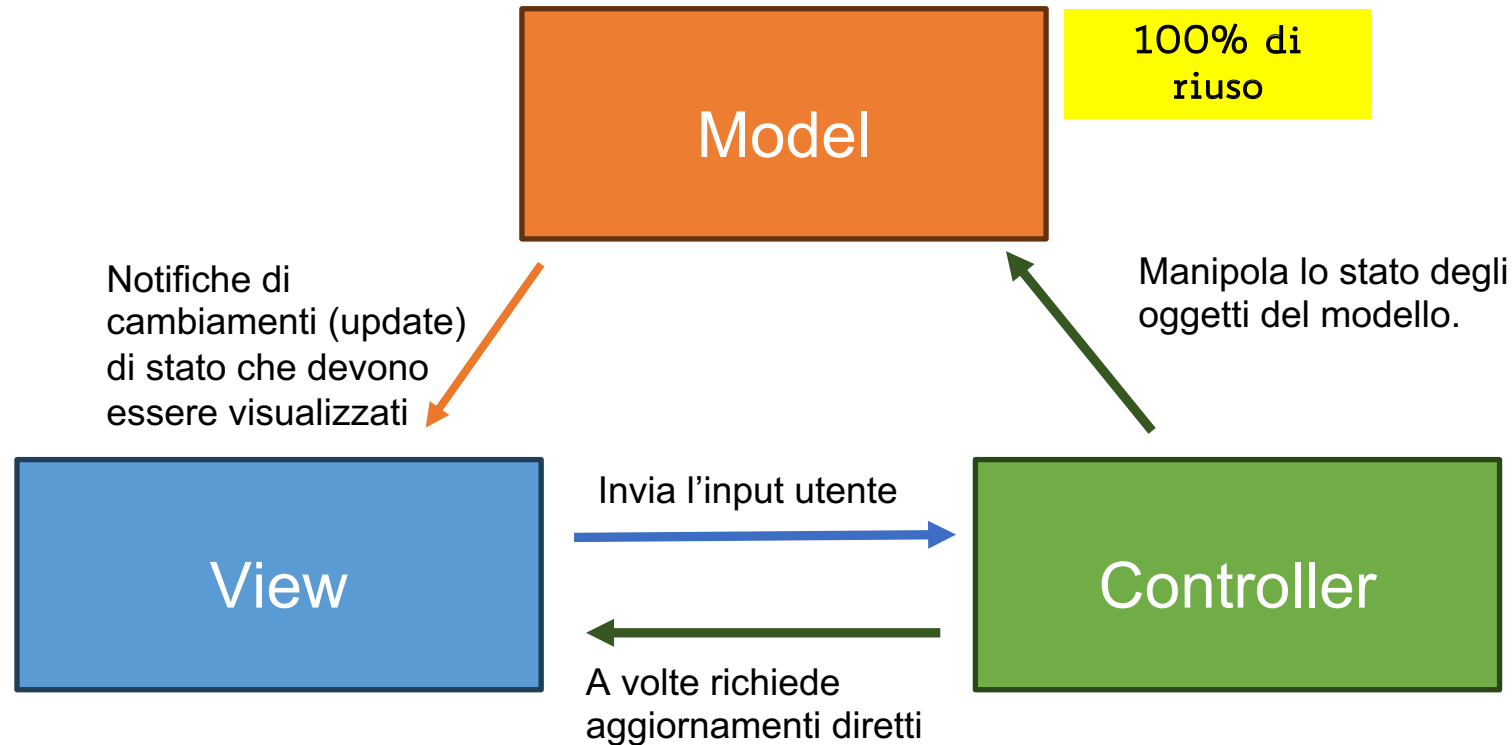
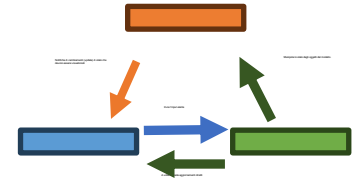
😊 Soluzione / Ricetta



- 1) Progettare consistentemente (rispetto al ruolo) il software in 3 package: model, view e controller.
- 2) Le classi del model si devono occupare solo di aspetti della logica
- 3) Le classi della view si devono preoccupare solo di aspetti di interfaccia
- 4) Le classi del controller diventano la sala di regia e controllano sia l'input utente che la manipolazione dello stato degli oggetti del modello

MVC e riuso del software 1

Le classi del Model devono essere riutilizzabili al 100%

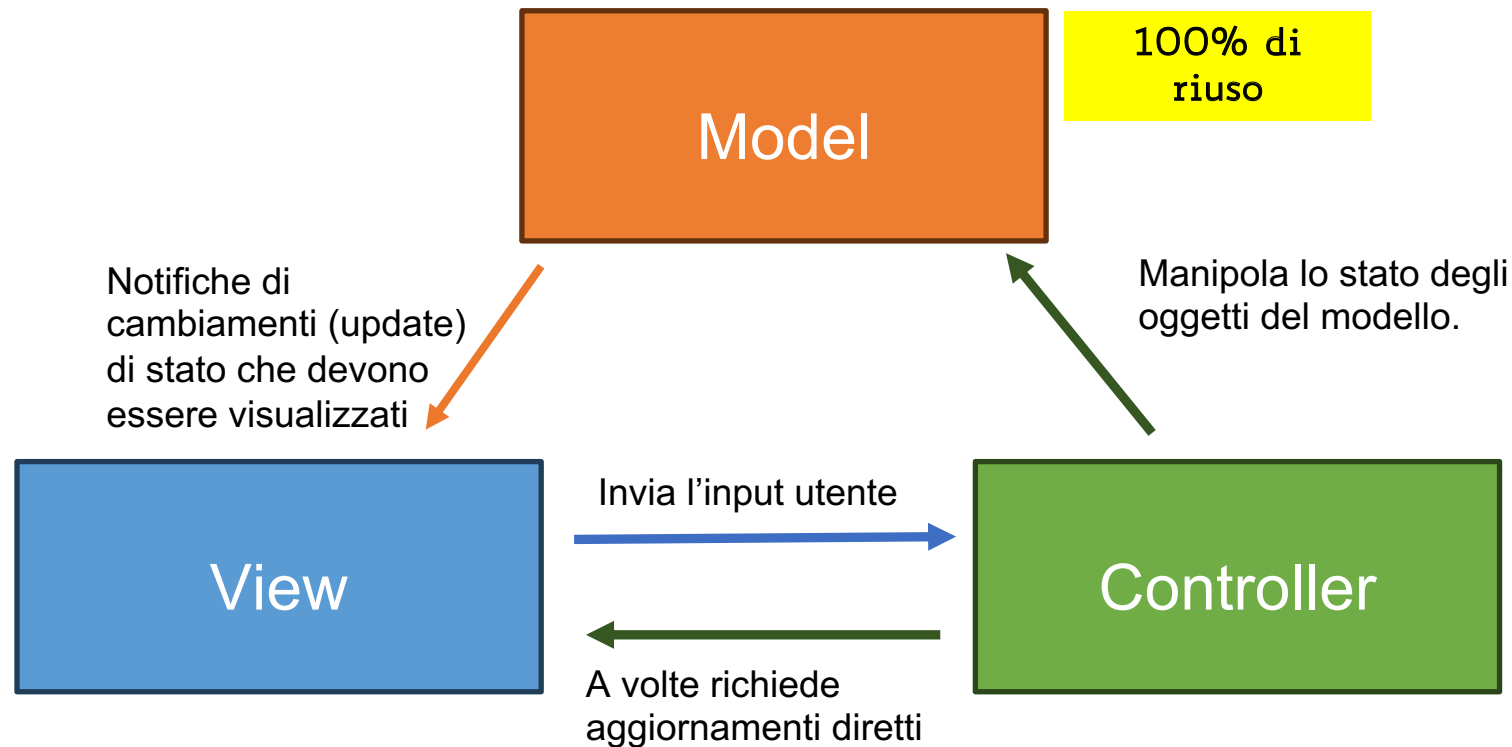
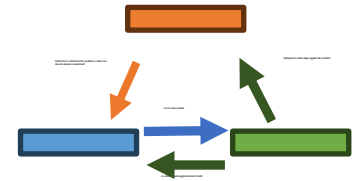


La tecnologia per realizzare la View (ad es. le librerie utilizzate) possono cambiare nel tempo.

Il Modello potrebbe essere in esecuzione su di una macchina remota (Server) dove non ci sono né schede video né schede audio.

MVC e riuso del software 2

Le classi del Model devono essere riutilizzabili al 100%



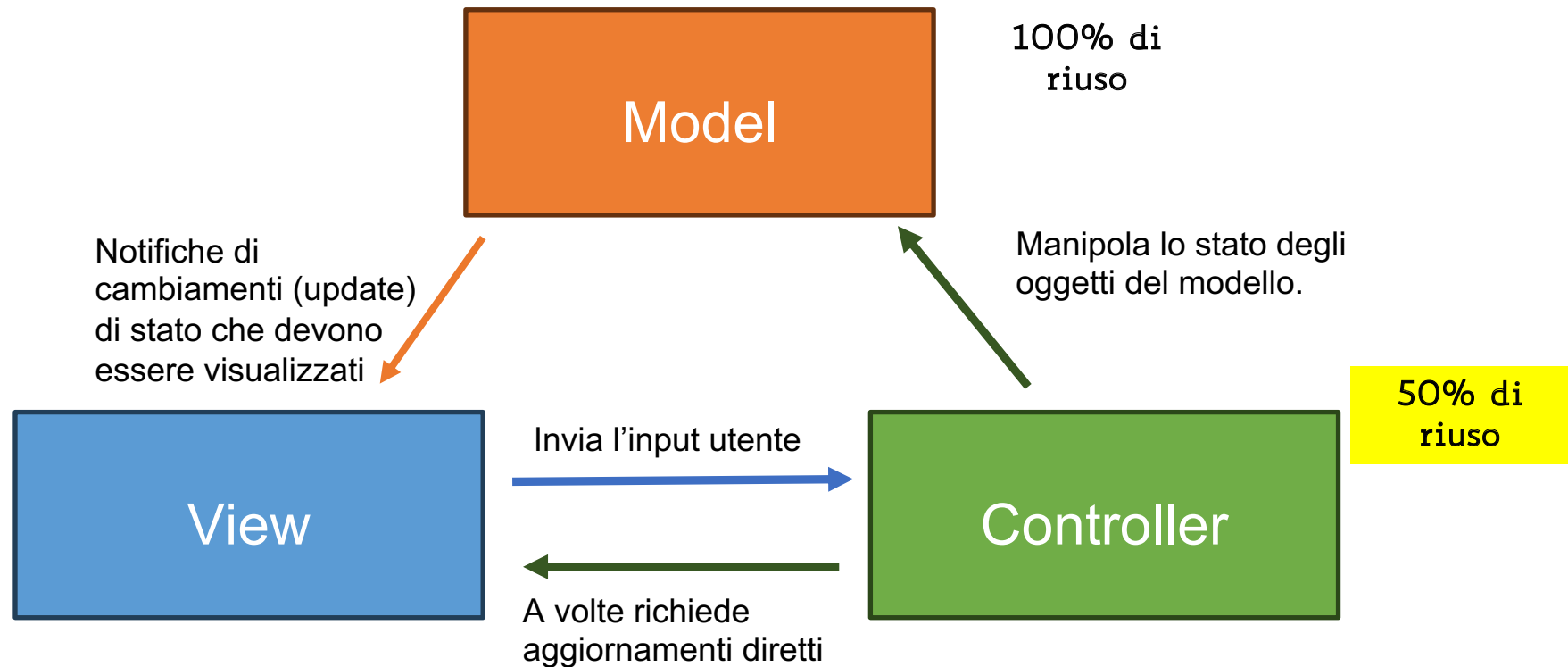
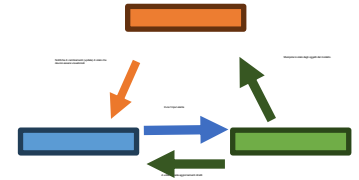
La tecnologia per realizzare la View (ad es. le librerie utilizzate) possono cambiare nel tempo.

Il Modello potrebbe essere in esecuzione su di una macchina remota (Server) dove non ci sono né schede video né schede audio.

Non ci devono essere dipendenze con classi di altri package né il modello si deve preoccupare di gestire asset grafici e audio.

MVC e riuso del software 3

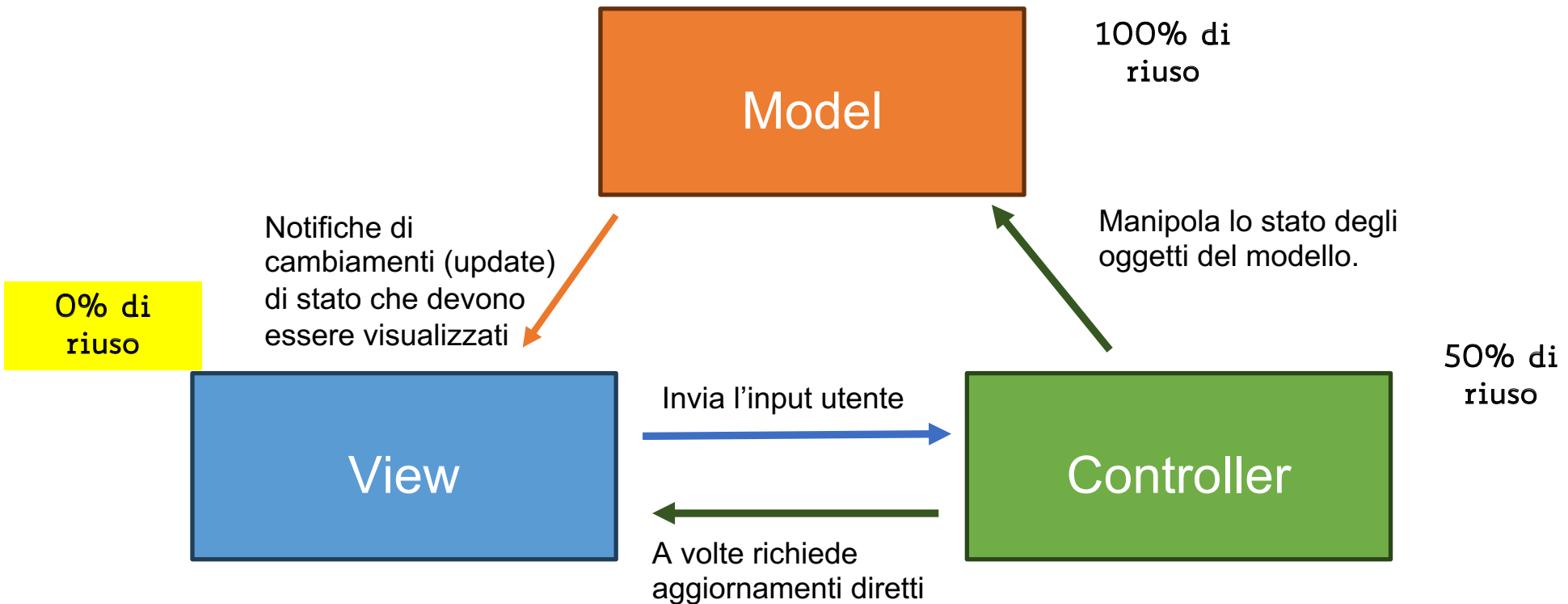
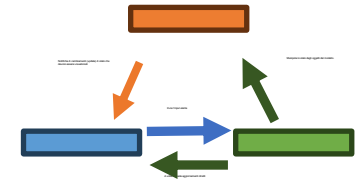
Le classi del Controller devono essere riutilizzabili al 50%



Il Controller deve poter essere riutilizzato al 50%, in quanto la parte che dipende dalla View non si può riutilizzare se la View cambia. Ovviamente il 50% che dipende dal Model non sarà riscritta.

MVC e riuso del software 4

Le classi del View hanno un grado di riuso pari a 0.



Sfortunatamente la View è l'interfaccia utente di uno specifico Modello e non potrà essere utilizzata in altre applicazioni. Inoltre la View dipende fortemente da tecnologie/librerie proprietarie o comunque dipendenti dalla piattaforma.

MVC e riuso del software 5

Model

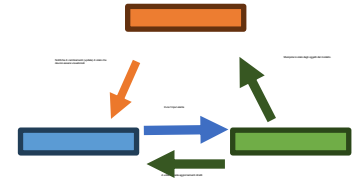
100% di riuso: lo posso riutilizzare a prescindere dalla tecnologia di View e posso anche collocarlo su una macchina remota, tanto rappresenta solo la logica e non gestisce asset grafici o audio.

Controller

50% di riuso: lo posso riutilizzare almeno per le parti del codice che interagiscono con il Modello

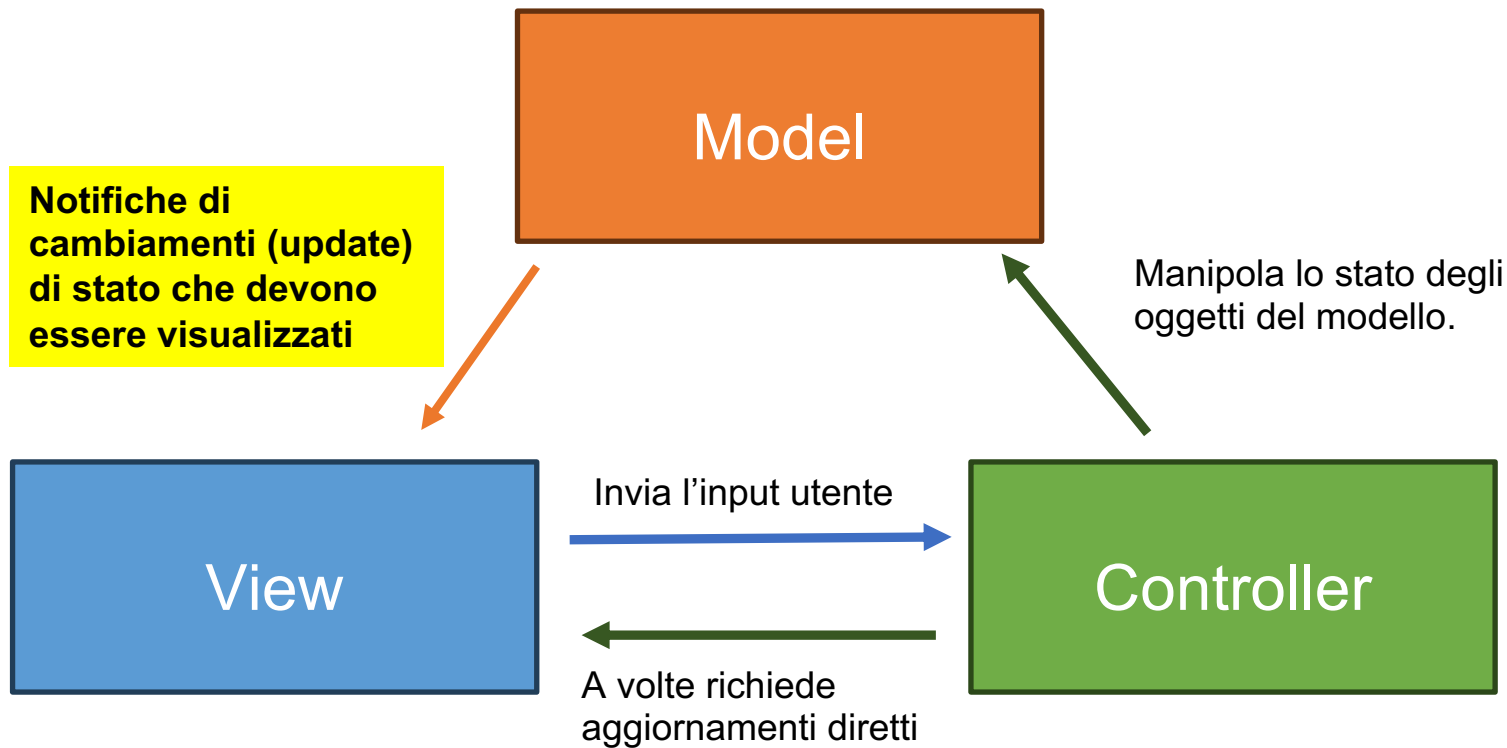
View

0% di riuso: non lo posso riutilizzare. Sicuro è il fatto che gli asset grafici e audio saranno gestiti dalla View e non sull'eventuale Server dove potrebbe essere in esecuzione il Modello.



MVC e riuso del software 6

Bisogna prestare molta attenzione su questo passaggio
"Notifiche di cambiamenti di stato"

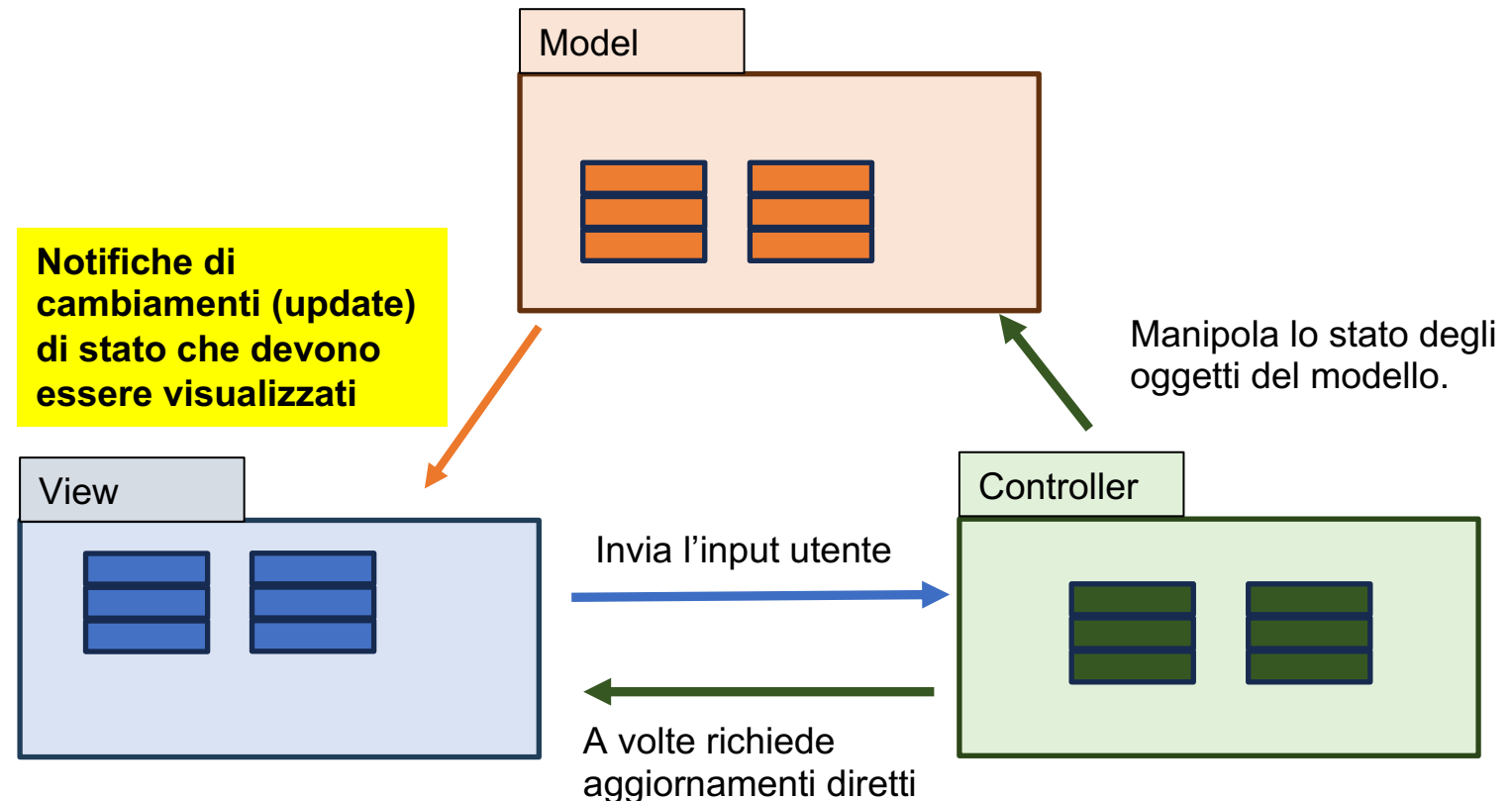


MVC e riuso del software 7

Bisogna prestare molta attenzione su questo passaggio

"Notifiche di cambiamenti di stato"

Guardiamo al progetto in termini dei 3 package



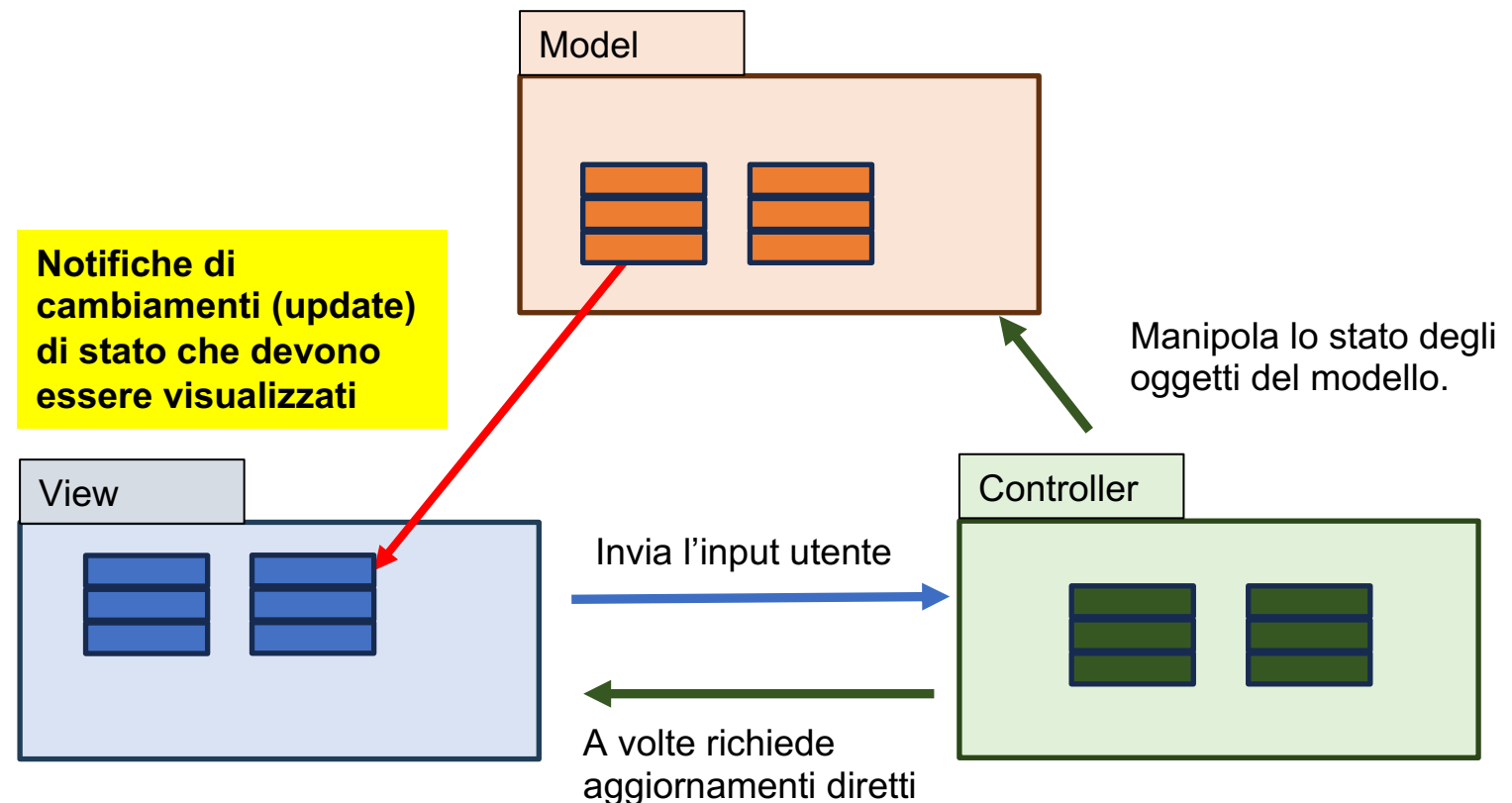
MVC e riuso del software 8

Bisogna prestare molta attenzione su questo passaggio

"Notifiche di cambiamenti di stato"

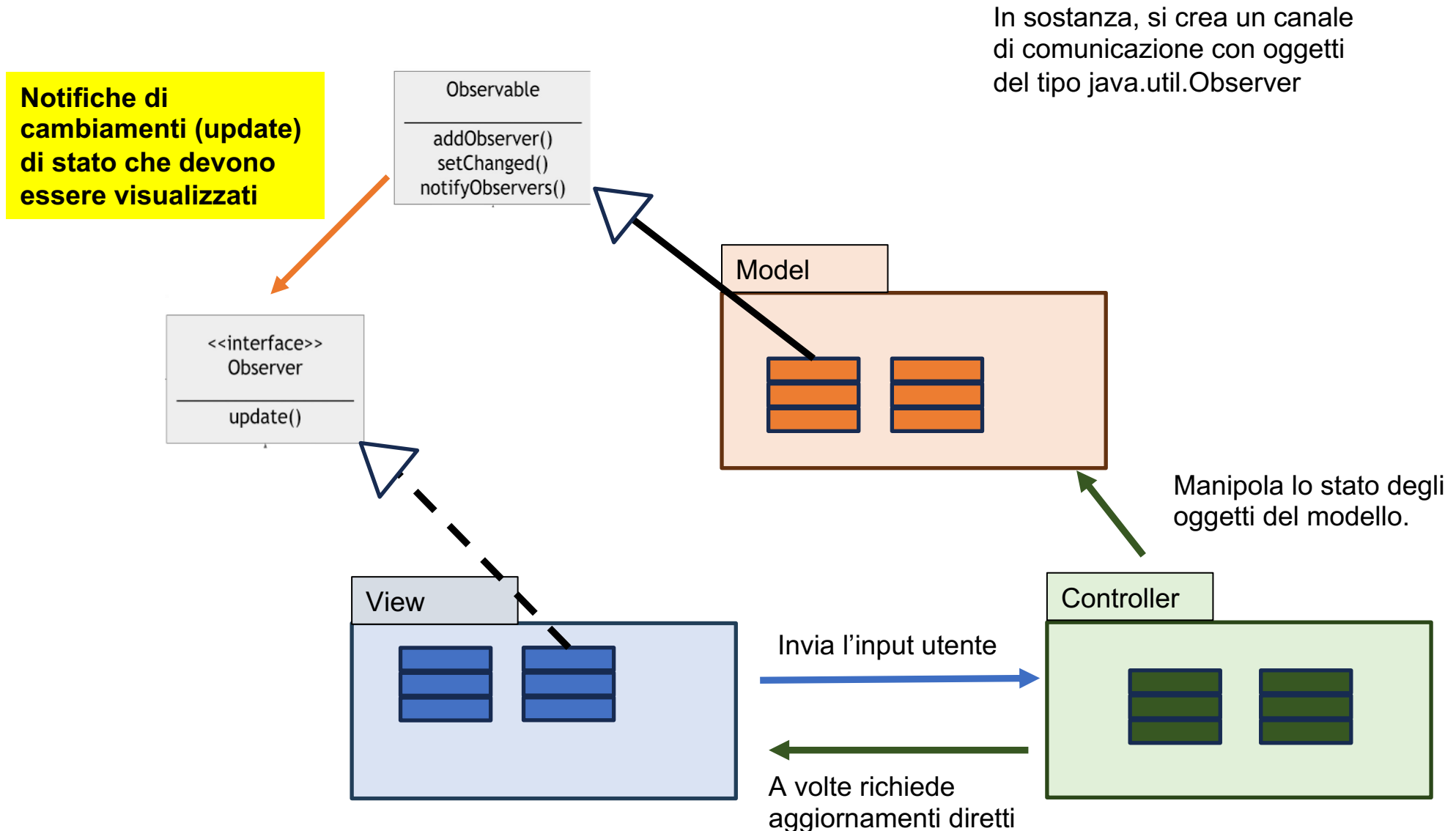
Guardiamo al progetto in termini dei 3 package

Richiamare da una classe del Model un qualche metodo di una classe della View rende il Model completamente DIPENDENTE DALLA VIEW! Abbattendo a 0% il suo grado di riuso del Model.



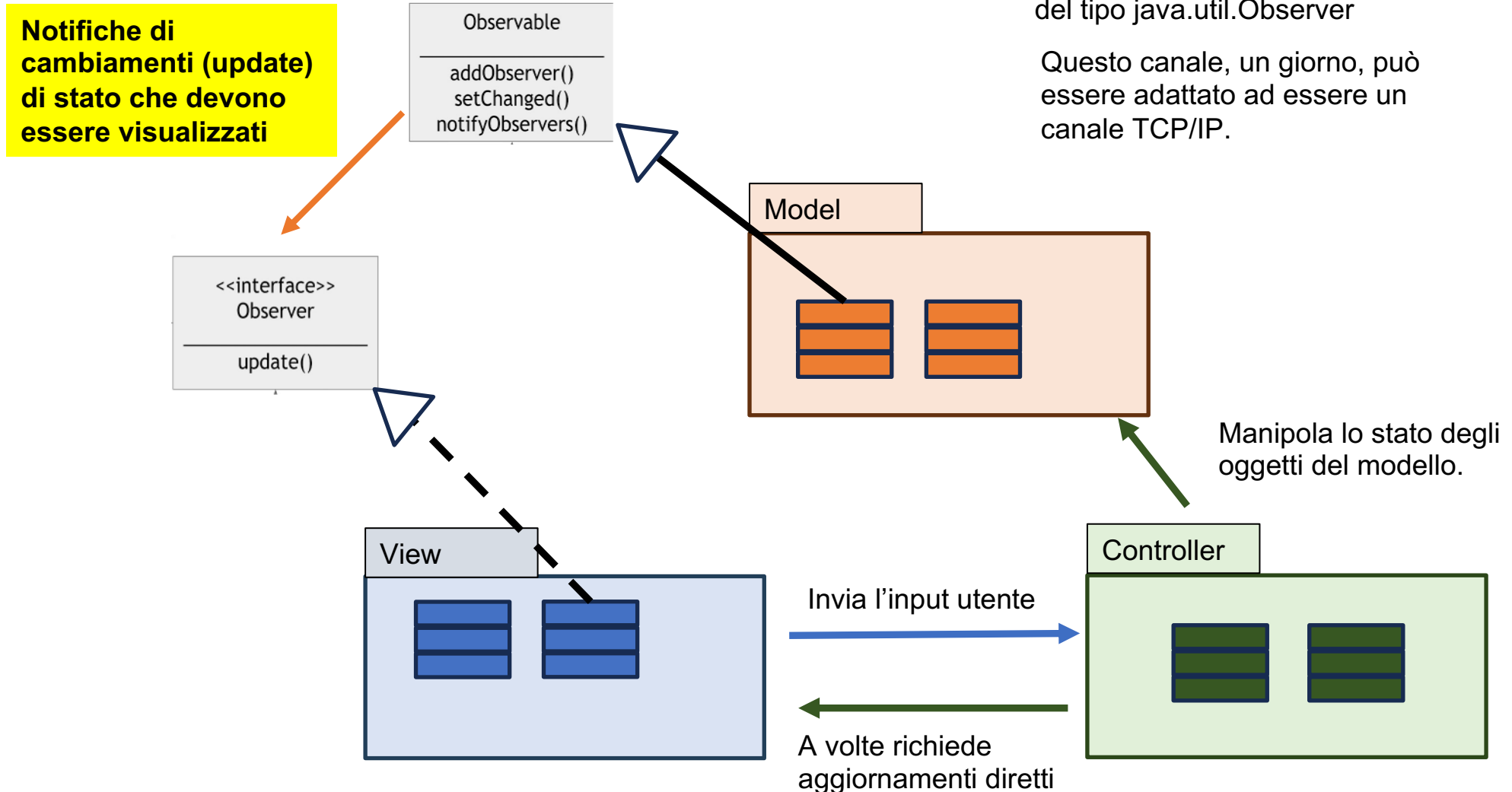
MVC e riuso del software 9

Separiamo le classi del Model da tutto il resto adottando il pattern Observer.

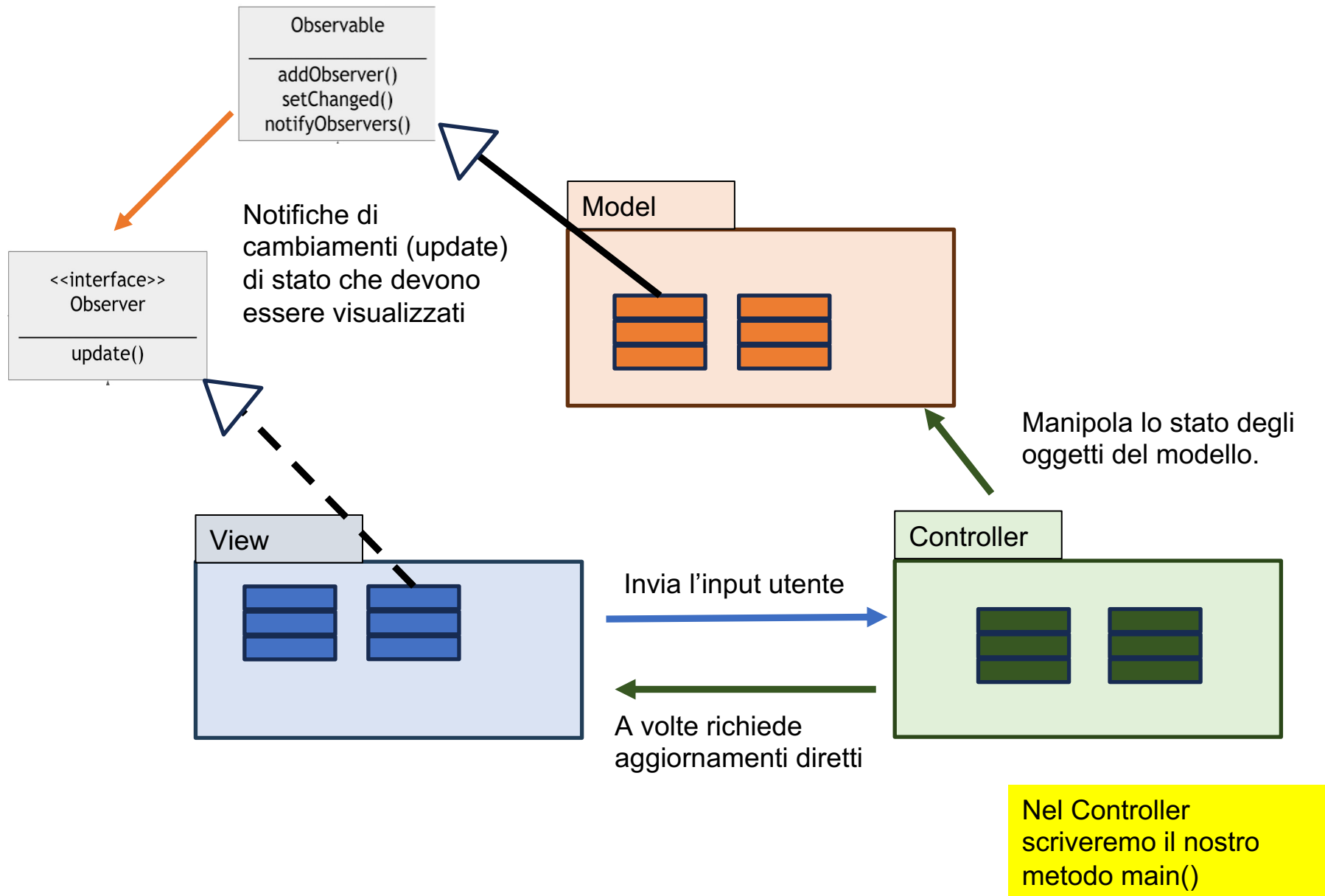


MVC e riuso del software 10

Separiamo le classi del Model da tutto il resto adottando il pattern Observer.



MVC + Observer



MVC + Observer + SWING

Nel Controller
scriveremo il nostro
metodo main()

```
public static void main(String[] args){  
  
    Model m=Model.getInstance();  
    View v=View.getInstance();  
    m.addObserver(v);  
  
    // Aggancia gli ActionListener ai  
    component della View  
    ....  
  
    // GAME LOOP  
    while (true)  
    {  
        ....  
        m.evolvi();  
        ...  
    }  
}
```

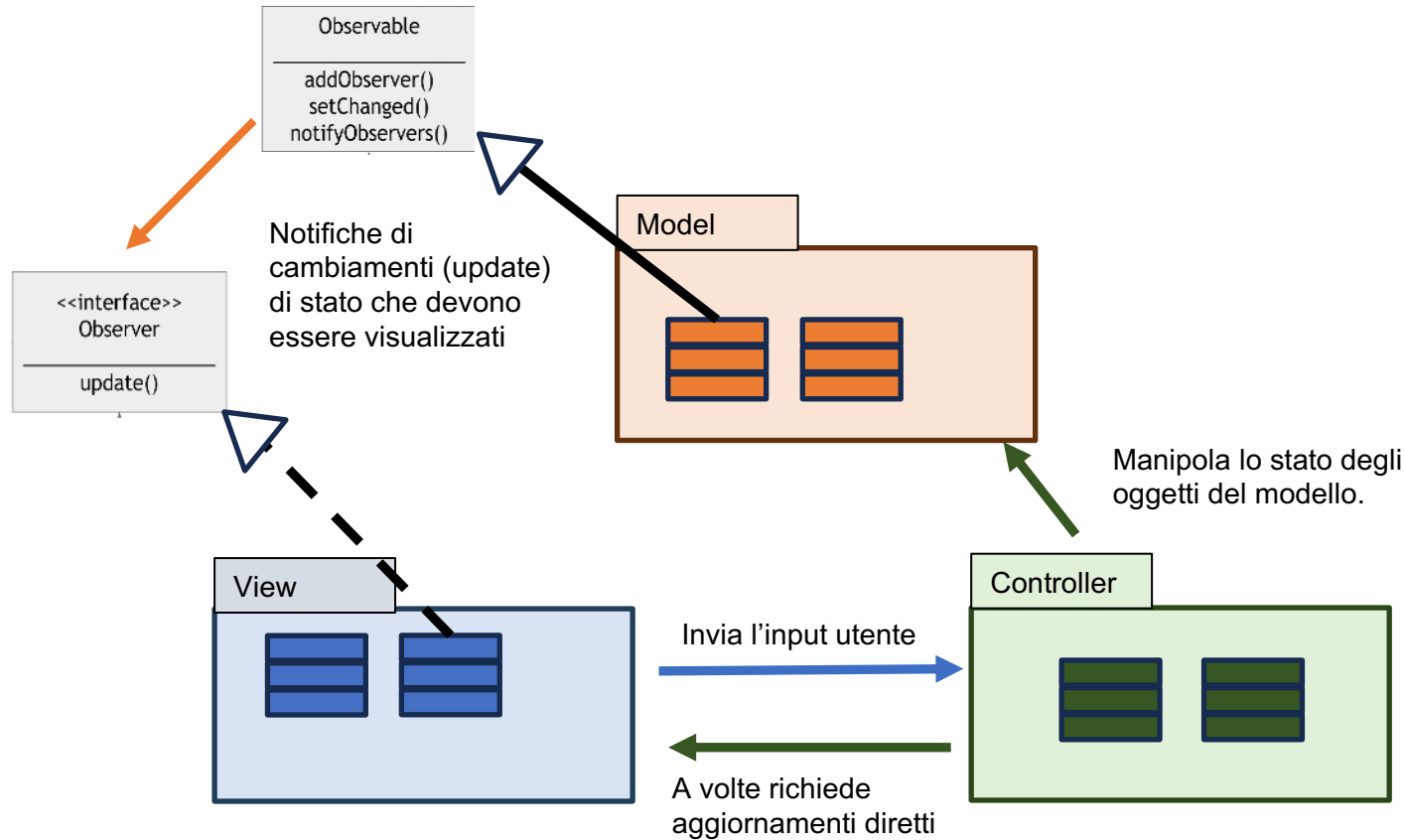
MVC + Observer

Nel Controller
scriveremo il
nostro metodo
main()

```
public static void main(String[] args){  
  
    Model m=Model.getInstance();  
    View v=View.getInstance();  
    m.addObserver(v);  
  
    // Aggancia gli ActionListener ai  
    component della View  
    ....  
  
    // GAME LOOP  
    while (true)  
    {  
        ....  
        m.evolvi();  
        ...  
    }  
}
```

Quando viene creata la
View, se si usano le
SWING, avremo la
creazione di un nuovo
Thread "Swing Event
Dispatch Thread"

MVC + Observer + SWING

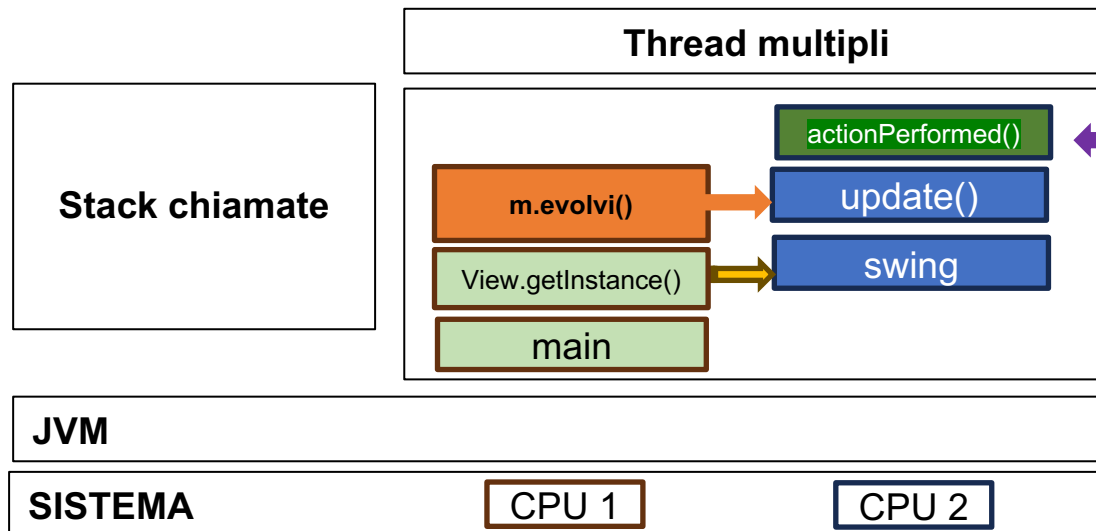


```
public static void main(String[] args){

    Model m=Model.getInstance();
    View v=View.getInstance();
    m.addObserver(v);

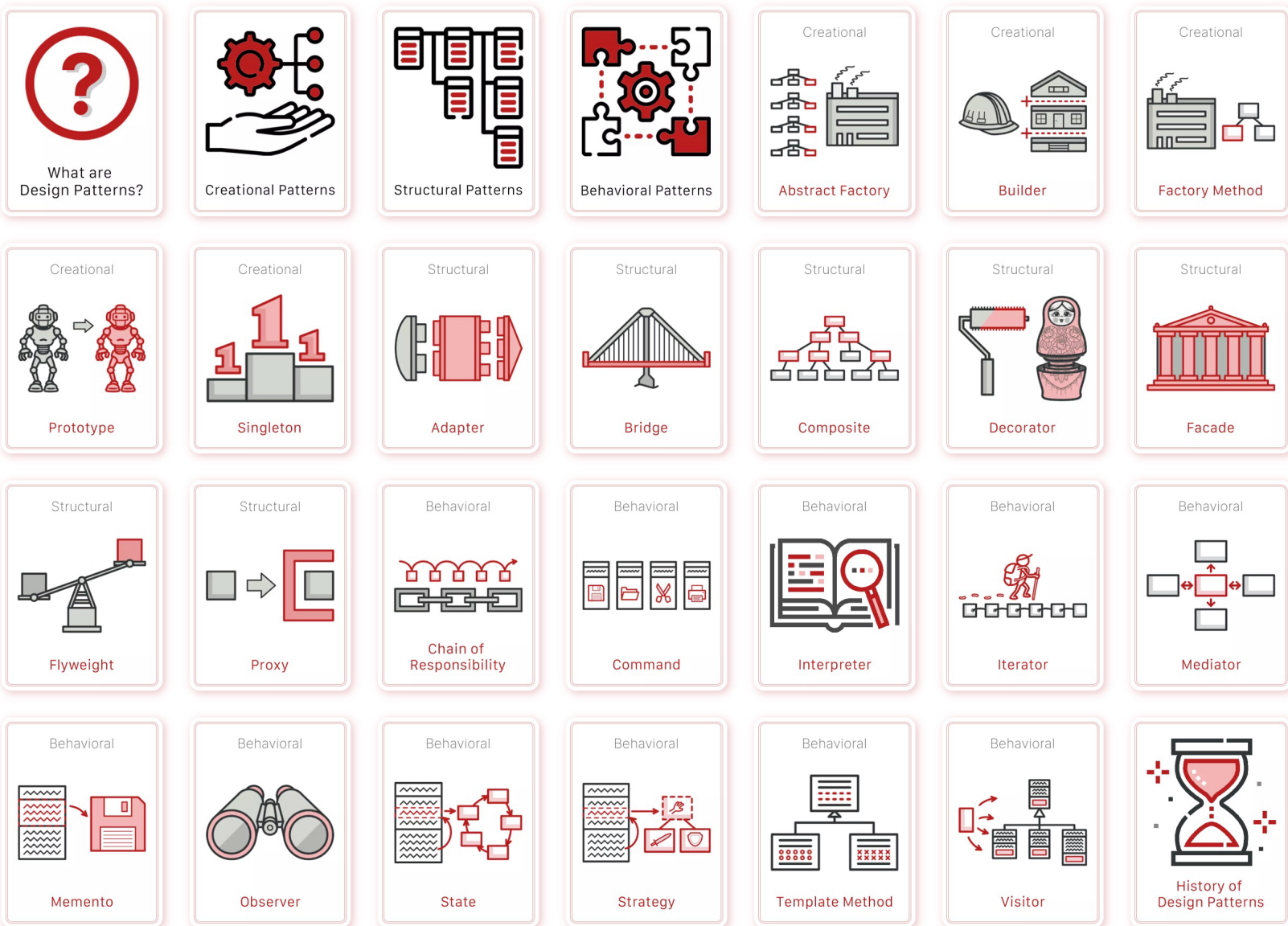
    // Aggancia gli ActionListener ai component
    della View
    ....

    // GAME LOOP
    while (true)
    {
        ....
        m.evolve();
        ...
    }
}
```



- ➡ Creazione ed avvio di un thread
- ➡ notifica
- ➡ evento
- Model
- View
- Controller/Game Loop

Esistono molti altri design pattern



Esistono molti altri design pattern

<https://design-patterns-101.vercel.app/>

